SIT 215

Assignment 3

Project

Report

s224326349
Angodavidanelage Rukshan Anthony Dias

Cover Page

Tasks	Completed
Task 1: Exploration of Automated Planning and PDDL (P/C)	☑
Task 2: 3 – minute video presentation (P/C)	☑
Task3: PDDL for the Wumpus World Project (D)	☑
Task 4: Solver Comparison (HD)	☑

Contents

Introduction to Automated Planning 3
Implementation Overview 3
Knowledge Base and Representation 10
Results and Discussions 14
Conclusion 22
Acknowledgment 22
References 22

Introduction to Automated Planning

One of the fundamental areas in artificial intelligence is none other than automated planning. It involves with creating action sequences that guides an intelligent agent from a specified stage to an intended state. This can be achieved by applying logical reasoning to possible sequences of action and their consequences in a particular setting. PDDL stands for Planning Domain Definition Language. This allows AI agents to model domains and set goals, plan action sequences in a format a machine can read. In this assignment we shall look into such codes in this study to investigate automated planning.

In this assignment, we first look into the *blocks_medium* domain and the *problem1.pddl* file. In here a robotic agent will stacks certain blocks to accomplish a target arrangement. Blocks are stacked, unstacked, picked up, and laid down according to logical predicates and rules that replicate an ordered world in which every action has a known result. By the end of this we shall grasp how PDDL guides agents in ordered decision making by encoding action frameworks, preconditions, and effects.

The assignment will be expanded by studying the *Wumpus problem* which is more complicated than *blocks_medium.* In this, the agent has to find the riches in the Wumpus World, which is a 4x4 grid simulation. The agent in Wumpus World uses local sensory inputs, such as the breeze and stink, to figure out that there are surrounding threats, compared to the simple Blocks World. In the end we compare the performance of the PDDL solvers from *planning.domains.* The problem was analyzed using the 2 solvers,

- BFWS FF Parser: A heuristic based forward search planner
- LAMA-first: A multi-heuristic A* planner that path finding using landmarks.

Implementation Overview

Blocks_medium

In here, from the pddlgym GitHub respository, I downloaded the blocks_medium.pddl and the problem1.pddl files to conduct this study. I chose this because it offers a really good planning challenge by converting an unstructured block configuration into an ordered stack by using a set of interconnected actions. The important physical restriction where it needs a free hand and only being able to move one block at a time are stored in the domain file. This arrangement matches actual robotic planning situations in which agents must successively reason through approved actions.

In these files we can a set of rules which defines the environment and the problem setup. In order to properly understand this implementation, it is very important that we try to understand the fundamental components of PDDL.

- **Predicates** - these are logical statements that characterize the current condition of events. Each and every predicate defines a condition. It could be true or false. We use these predicates to establish preconditions and effects in actions. We can also use them to define the start and target states of this task. Some examples will be listed below. When planning, these predicates are changed by activities and help as models for current situation of events.

- **Actions** – These are what the agent is capable of doing and are defined with :action. Every action consist of,
  - :parameters – the variables that are used as input by the action
  - :precondition – the prerequisites that must be met in order for the action to be carried out.
  - :effect - The world's changes following the execution of the action

- **Objects** – These are the real world examples used in planning such as blocks a, b, and c that are declared in the problem file.

- **Domain File** – In this assignment, the file is blocks_medium.pddl. It comprises of predicates we mentioned above. It has 4 main actions such as pickup, putdown, stack, and unstack. They all come with certain preconditions and effects.

Let's look at the actions in the Domain file,

(:action **pick-up** :parameters (?x - block) :precondition (and (clear ?x) (ontable ?x) (handempty)) :effect (and (not (ontable ?x)) (not (clear ?x)) (not (handempty)) (handfull) (holding ?x)))	The above code implements the schema for picking up a block from the table. The preconditions say that block ?x has noting on top, block ?x is on the table, and the robots hands are empty. Then it is instructed to hold the block, and we declare that the block is no longer on the table. We say that the held blocks aren't clear and that the hands of the robot is no longer empty.
(:action **put-down** :parameters (?x - block) :precondition (and (holding ?x) (handfull)) :effect (and (not (holding ?x)) (clear ?x) (handempty) (not (handfull)) (ontable ?x)))	With this action the robot is able to putdown the block it is holding. In here the precondition is that the robot should be holding a block. Then, it is instructed to place it on the table and it goes on to declare that the robots hand are empty and its no longer holding any block.
(:action **stack** :parameters (?x - block ?y - block) :precondition (and (holding ?x) (clear ?y) (handfull)) :effect (and	This code places a block on top of another clear block . The preconditions are that robot should be holding ?x and block ?y has no blocks on top of it. Then it is instructed to place block ?x on top of ?y. It is then declared that block ?x is clear, the robots hand are free, block ?y is no longer clear, and that the robot is no holding ?x anymore.

(not (holding ?x)) (not (clear ?y)) (clear ?x) (handempty) (not (handfull)) (on ?x ?y)))	
(:action **unstack** :parameters (?x - block ?y - block) :precondition (and (on ?x ?y) (clear ?x) (handempty)) :effect (and (holding ?x) (clear ?y) (not (clear ?x)) (not (handempty)) (handfull) (not (on ?x ?y)))	In this code the robot can remove a block which is on top of another block by following the instructions in the code. The preconditions in here are block ?x is stacked on block ?y and nothing is on top of ?x and the robots hand should be empty. Then the Robot will pick up block ?x and clear block ?y. it is then declared that ?x is no longer on top of ?y and the held block is not clear since it is in the robots hand and they are occupied.

For an example, a stack needs the target block to be clear and that the robot should be holding a block. In this execution, the new positions and hand status are reflected in the updated KB. The planner can choose steps that bring the goal closer by simulating state changes with these activities.

Even if the steps aren't given during the planning process, the agent will anyways choose the order of action. For example, if the planner wants to put block b1 on b3 when it is initially on top of b2. It must first,

1. Make sure that b1 is clear and the robots hands are empty
2. Then perform the 'unstack' action to remove b1 from b2
3. Now the robot is (holding b1), and its hands are full (handful)

The planner also makes sure that the restrictions are followed when executing. We can see this in step 1 and 2. The knowledge base is quickly updated to represent the current action taken place. If the action is,



- (clear b1) is added.
- (on b0 b1) is removed.
- (holding b0) is added.
- (handempty) is removed, and (handfull) is added.

These adjustments give the planner the ability to decide what to do next while making sire all the prerequisites are met for the next action. The planner will automatically move towards the target/final stack configuration using logical reasoning from the KB.

This shows the basic concept of symbolic AI, which controls symbolic representations of the environment to accomplish set objectives through the application of structured logical reasoning.

Wumpus World - D

PDDL Vocabulary is a widely used language for expressing planning issues. We use that again in Wumpus world project to design an intelligent agent that could go around a grid based environment, avoid obstacles, collect the gold, get rid of any Wumpus if needed, and safely return to the starting place. This is the primary goal of this project. Each cell in the environment's 4x4 model represents a distinct state where it could be empty, have a pit, the Wumpus, or the gold.

Annotated code implementations and solutions for the identified issues.

There were several logical and structural problems that prevented optimal path finding and hazard handling that I found while investigating the given pddl files. Below is a list of those problems and how I implemented the solution.

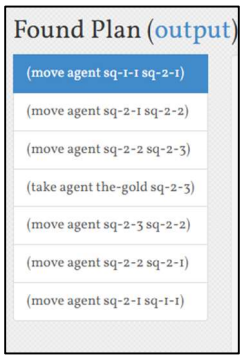
Problem 1: improper movement logic

Any object, even the gold, may 'move' like an agent in the original implementation. In that code, the planner tried to move the gold on its own which is unrealistic, as a result of this logical error.



Solution:

By including (agent?who) in the preconditions, I limited the move operation to an agent specific logic. This makes sure that only the agent can move about the grid.




```
(:action move
  :parameters (?who ?from ?to)
  :precondition (and (adj ?from ?to)
                     (not (pit ?to))
                     (at ?who ?from)
                     (not (dead ?who))
                     (agent ?who))
  :effect (and (not (at ?who ?from))
               (at ?who ?to))
)
```

The 'move' action enables the agent to move from one spot to another if the cells are adjacent, if the cell doesn't have a pit, and the agent is alive.

It also checks whether the agent is actually an agent (agent ?who). once it moves, the agents location is updated to the new cell.

Problem 2: Agents death is not specified

The agent's death logic is not considered in the initial pddl in case it encounters any pit. The agent could enter a pit and then move on as if nothing happened. This is contrary to the Wumpus World's logical framework.

Solution:

We introduced fall as a new action. This action immediately marks the agent as dead and stops them from moving any more if it encounters a pit.

```
(:action fall
  :parameters (?who ?where)
  :precondition (and (at ?who ?where)
                (pit ?where)
                (not (dead ?who)))
  :effect (and (dead ?who))
)
```

In this instance the fall actions is dealt with the situation in which the agent enters a pitted cell. According to the pre-conditions, the agent (?who) needs to be at the place (?where) there must be a pit there (pit?where), the agent isn't already dead (not(dead?who)). If these requirements are met, the actions results in to mark the agent as dead (dead?who). The agent will not move again.

Problem 3: Absence of Wumpus Encounter Logic

In the beginning, the agent might enter the Wumpus and face no issue, which is not realistic for the hazard model of the problem.

Solution:

I included an encounter action that starts if the agent enters the Wumpus's cell without firing a shot. In accordance with the danger logic, this action indicates the agent as dead.

```
(:action encounter
  :parameters (?who ?where)
  :precondition (and (at ?who ?where)
                (at wumpus ?where)
                (wumpus_alive)
                (not (dead ?who)))
  :effect (and (dead ?who))
)
```

In the instance when an agent runs into the Wumpus, we represent it by encounter action. Only when the wumpus is still living (wumpus_alive), the agent (?who) and the wumpus are in the same cell (?where), and when the agent is not already dead (not(dead?who)) does the action start. This will result in the agent being tagged as dead (dead?who), which stops it from moving further.

Problem 4: Reusability of arrows

The agent was able to shoot several times with a single arrow in the first area. That cannot happen.

Solution:

After shooting we marked the arrow as spent by adding a predicate (used?arrow).

```
(:action shoot
  :parameters (?who ?where ?arrow ?victim ?where-victim)
  :precondition (and (have ?who ?arrow)
                     (at ?who ?where)
                     (at ?victim ?where-victim)
                     (adj ?where ?where-victim)
                     (not (used ?arrow)))
  :effect (and (dead ?victim)
               (not (at ?victim ?where-victim))
               (not (have ?who ?arrow))
               (used ?arrow)
               (not (wumpus_alive)))
)
```

The shoot action here is to take down the wumpus if it's in an adjacent cell. For this to take place there are several pre-conditions. The agent must be equipped with an arrow (have ?who ?arrow), it should be in a cell (at ?who ?where), the wumpus (?victim) must be in an adjacent cell (adj ?where ?where-victim), and the arrow shouldn't be used before (not (used ?arrow). Once these conditions are met, the arrow is taken from the agent's possession (not (have?who?arrow)), the Wumpus is marked as dead (dead?victim), it is taken out of its cell (not (at?victim?where-victim)), it is marked as used (used?arrow), and it is proclaimed dead (not (wumpus_alive)). The agent is only allowed to take a shot only once.

Problem 5 Incomplete adjacencies

The movement of the agent across the grid was limited by the fact that many squares were not mutually reachable. Without a safe route or a plan of action, the gold at (sq-2-3) was inaccessible because of adjacent that were not well connected and/or because pits and the Wumpus restricted it. Also, the Wumpus was alive and in direct danger, yet there was no safe or obvious way to shoot it or get away from it. I made a version of the domain/problem files that was simplified and corrected

```
(:init
 ;; Vertical adjacencies notation: sq-{X-axis-coordinate}-{Y-axis-coordinate}
 (adj sq-1-1 sq-1-2) (adj sq-1-2 sq-1-1)
 (adj sq-1-2 sq-1-3) (adj sq-1-3 sq-1-2)
 (adj sq-1-3 sq-1-4) (adj sq-1-4 sq-1-3)
 (adj sq-2-1 sq-2-2) (adj sq-2-2 sq-2-1)
 (adj sq-2-2 sq-2-3) (adj sq-2-3 sq-2-2)
 (adj sq-2-3 sq-2-4) (adj sq-2-4 sq-2-3)
 (adj sq-3-1 sq-3-2) (adj sq-3-2 sq-3-1)
 (adj sq-3-2 sq-3-3) (adj sq-3-3 sq-3-2)
 (adj sq-3-3 sq-3-4) (adj sq-3-4 sq-3-3)
 (adj sq-4-1 sq-4-2) (adj sq-4-2 sq-4-1)
 (adj sq-4-2 sq-4-3) (adj sq-4-3 sq-4-2)
 (adj sq-4-3 sq-4-4) (adj sq-4-4 sq-4-3)

 ;; Horizontal adjacencies notation: sq-{X-axis-coordinate}-{Y-axis-coordinate}
 (adj sq-1-1 sq-2-1) (adj sq-2-1 sq-1-1)
 (adj sq-2-1 sq-3-1) (adj sq-3-1 sq-2-1)
 (adj sq-3-1 sq-4-1) (adj sq-4-1 sq-3-1)
```

I created a compact 3x2 grid with full neighbors and made sure there was at least one secure, Wumpus-free route to get the gold and go back.

Following the implementation of these fixes, I used the planning.domains to test the issue.

Wumpus World Constraint Solver Analysis -HD

The PDDL domain and issue files for the Wumpus World environment are developed in depth in this part. We discussed what the goal in this Wumpus world scenario is, and we fixed the logical errors in the .pddl files.

In this section we incorporate **Sensing** into the domain to stop the agent from moving mindlessly into unknown spaces. In the previous file we only had move, take, shoot, fall, and encounter actions. But in here we add the **Sense** action. In a real world simulation, the agent must detect its surroundings before moving in order to avoid unknown risks, whereas in a standard Wumpus world, the agent moves optimally if it has complete information of the grid.

Logic behind implementation

- Before it can move, the agent can determine whether a grid site is safe thanks to the sense action.
- The predicate **(sense?square)** is representation of this and is necessary for any move action to occur.
- Exploring the grid blindly is prohibited by this reasoning, which is similar to the real world navigation techniques we use. So the agent will stay out of unexplored territory.

I implemented the following constraints as shown in the domain pddl below.

```
(:action sense
   :parameters (?who ?from ?to)
   :precondition (and (adj ?from ?to)
                      (at ?who ?from)
                      (agent ?who))
   :effect (sensed ?to)
)

(:action move
   :parameters (?who ?from ?to)
   :precondition (and (adj ?from ?to)
                      (not (pit ?to))
                      (at ?who ?from)
                      (not (dead ?who))
                      (sensed ?to)
                      (agent ?who))
   :effect (and (not (at ?who ?from))
                (at ?who ?to))
)
```

This make sure that the agent will not make a move without first confirming the target square's safety. This is very important which aids in planning stability and dependability by preventing pits and Wumpus encounters

Solver Analysis -HD

Comparative analysis for the solvers.

Aspect	BFWS FF Parser	LAMA-First
Heuristic Method	Only the immediate next step is optimized by the FF heuristic.	Multi-heuristic, landmark-based A* that optimizes internationally
Execution Time	Slow	quick
Path finding Efficiency	same	same
Stability	More chances to become lost or take an unnecessary route	Solid and steady when determining the best course of action

Because of its optimal heuristic guidance, LAMA-First took less time to show the plan. A direct route was followed with a bit more steps because of the sense action. The BFWS ff parser took a bit more time because it added needless backtracking which lengthened the execution time without changing the overall number of steps (10).

Knowledge Base and Representation

Blocks_medium

The domain file *blocks_medium.pddl* contains predicates that define the Knowledge Base (KB). They comprise of both static and dynamic connections between the agent's state and blocks.

(clear ?x)	Block ?x has no other block on top.
(on ?x ?y)	Block ?x is on top of block ?y
(ontable ?x)	Block ?x is directly on the table
(Holding ?x)	The robot is currently holding block ?x
(handempty)	The robot is not holding anything.
(handful)	The robot is currently carrying a block

These predicates are used in the problem file to set the start and target goal states. The ***:goal*** section sets the target stack structure that the planner must reach, the ***:init*** section describes the initial block layout and the robot's current state. The planner will then make decisions during execution by following a series of steps that meet all preconditions and move the KB's state closer to achieving our target structure. Thanks to this, the agent does not have to carry out predefined action but rather, it creates a plan on the spot by deciding what has to be done to change the world state from its starting configuration to the targeted one.

The problem1.pddl file defines 19 block objects: from b0 to b18. It shows the starting state and the target we want to achieve.

Initial / starting stack

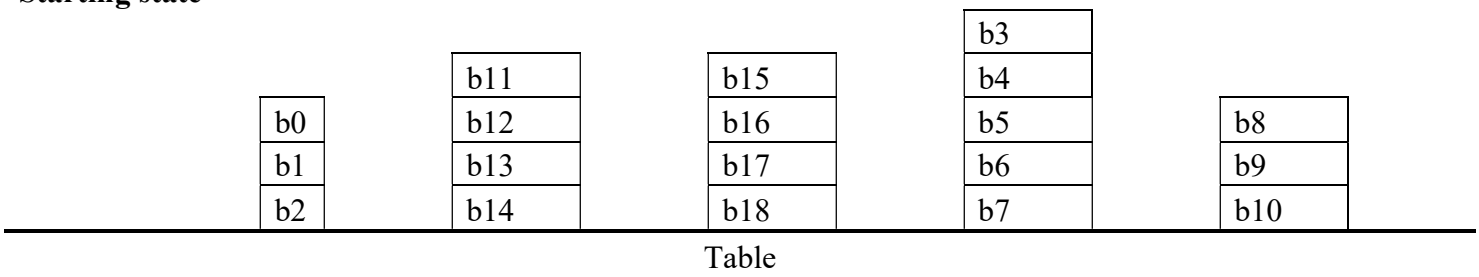
```
(define (problem generatedblocks) (:domain blocks)
  (:objects
       b0 - block
    b1 - block
    b10 - block
    b11 - block
    b12 - block
    b13 - block
    b14 - block
    b15 - block
    b16 - block
    b17 - block
    b18 - block
    b2 - block
    b3 - block
    b4 - block
    b5 - block
    b6 - block
    b7 - block
    b8 - block
    b9 - block
  )
```

```
  (:init
    (clear b0)
    (clear b11)
    (clear b15)
    (clear b3)
    (clear b8)
    (handempty)
    (on b0 b1)
    (on b11 b12)
    (on b12 b13)
    (on b13 b14)
    (on b15 b16)
    (on b16 b17)
    (on b17 b18)
    (on b1 b2)
    (on b3 b4)
    (on b4 b5)
    (on b5 b6)
    (on b6 b7)
    (on b8 b9)
    (on b9 b10)
    (ontable b10)
    (ontable b14)
    (ontable b18)
    (ontable b2)
    (ontable b7)
  )
```

```
  (:goal (and
    (on b4 b8)
    (on b8 b5)
    (on b5 b1)
    (on b1 b17)
    (ontable b17)
    (on b0 b15)
    (on b15 b13)
    (ontable b13)
    (on b2 b6)
    (on b6 b9)
    (on b9 b7)
    (ontable b7)
    (on b11 b12)
    (on b12 b18)
    (on b18 b10)
    (ontable b10)
    (on b16 b3)
    (on b3 b14)
    (ontable b14)))
)
```

Target stack

Starting state



Table

The above diagram represents the blocks and how they are placed on the table at first. Blocks b0 and b1 are placed on top of each other in the initial state. We can also see that block b2, b7, b10, b14, and b18 are places on the table. The PDDL file's ***:init*** section describes the starting state. There is nothing on top of the blocks b0, b11, b15, b3, and b8. It is clearly given in the pic above. The predicate 'handempty' means that the robot's hand is empty.

In here the goal section describes the new order which the blocks need to be stacked and rearranged,

For example,

- b4 comes on b8, which is on b5. b5 should be on b1, and b1 on b17, which is on the table itself.

- b0 is on b15, where b15 is placed on b13
- Similarly, it follows the rest of the steps to achieve the target stack.

The below table shows the knowledge base state changes for every important actions in blocks_medium. Every action changes the state. By taking that into consideration, the hand status, block positions, and clear or unclear status are modified.

Action	**Before state**	**After state**
Pick-up (b2)	(ontable b2), (clear b2), (handempty)	(holding b2), (not (ontable b2)), (not (clear b2))
Unstack (b0, b1)	(on b0 b1), (clear b0), (handempty)	(holding b0), (clear b1), (not (on b0 b1)), (handfull)
Put-down (b3)	(holding b3), (handfull)	(ontable b3), (clear b3), (handempty), (not (handfull))
Stack (b3, b11)	(holding b3), (clear 11), (handfull	(on b3 b11), (clear b3), (handempty), (not (clear b11))

Wumpus World - D

The Wumpus World Project knowledge base is an organized display of the surroundings and the agent's condition within them. The environment is represented as grid, with the Wumpus, the gold, a pit, or a safe cell in each cell. Logical predicates are used to track the position of the agent, its arrows and gold, and the state of dangers like whether the Wumpus is alive or dead.

The main predicated used here are,

(adj ?square-1 ?square-2)	Adjacency relationships
(pit ?square)	Locations of pits
(at ?what ?square)	Tracks position of agent, Wumpus, or gold
(have ?who ?what)	Tracks what the agent is holding
(dead ?who)	Represents if the agent or Wumpus is dead
(used ?arrow)	Marks if the arrow has been used
(wumpus_alive)	Checks if the Wumpus is still alive
(agent ?who)	Identifies the agent in the domain
(victim ?who)	Target in the 'shoot' action. (Wumpus)
(arrow ?arrow)	To label the arrow as an object for shooting

The initialization of the goal state happens at the ***:init***. It initializes the world state. It determines the starting position of the agent, where the Wumpus and the gold are placed, and where the pits are located. The agents movements are defined by adjacencies which are declared clearly. The agents start with the arrow, represented as (have agent the-arrow). The ***:goal*** specifies the intended end state where the agent needs to return to the starting position (sq-1-1) with the gold in hand.

The Wumpus world problem is a good example of symbolic reasoning in artificial intelligence, where logical predicates stand in for actual objectives and dangers. The agent utilizers the knowledge base in PDDL based planning to create safe routes guiding to the gold and to stay away from well-known pit areas. It is also used to retrieve the gold and find the way back to the beginning using just logical progression and symbolic references.

Wumpus World Constraint Solver Analysis -HD

The sensing constraint has a significant influence on the creation and updating of the agents knowledge. The predicate **(sense?square)** is added to the grid's state when the agent performs a sense action. This shows that the agent has new knowledge about that specific grid space. To make sure the knowledge base is in line with the condition of the environment, the agent cannot execute a move action until the square is sensed. This is something similar to a scenario I've seen in the 'Fog of war' game design. In here, the agent only knows about the locations it has visited, this too forbids exploring unsure areas.

That is why the agent only sense the blocks once and after collecting the gold it didn't have to sense because those spaces were already marked as safe.



The addition of the sense action made the agent update its knowledge base dynamically and increase its understanding before moving. By avoiding pits and unsafe spaces, the agent uses this sensing to move along safe routes.

```
:precondition (and (adj ?from ?to)
                   (at ?who ?from)
                   (agent ?who))
```

The logical precondition used here helps in maximizing path finding and provides safety by not allowing the agent to move into any danger. The agent's awareness is updated by each sensed square, preventing dangers and allowing the agent to make smart choices.

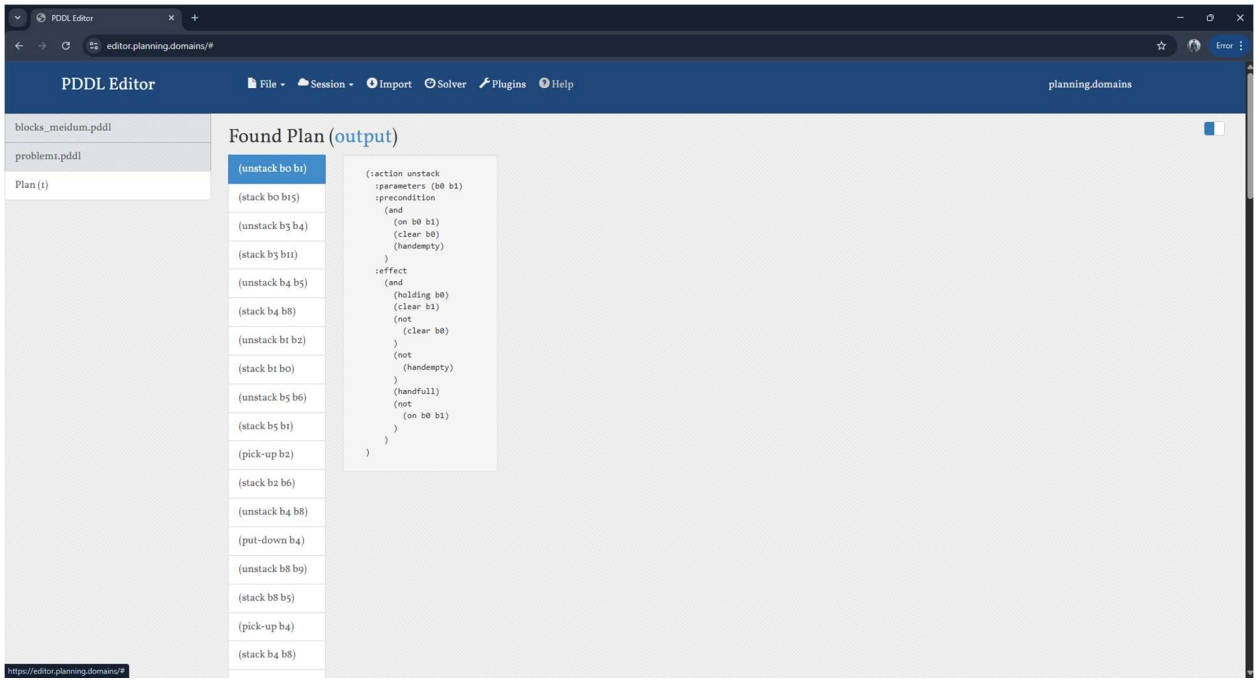
```
32                  (sensed ?to)
33                  (agent ?who))    ;;  Only the agent can move
```

In the move action, the agent can move only if it senses a square. That is what's told in this code.

Results and Discussions

Blocks_medium

I used the online planner at https://editor.planning.domains/ to run *blocks_medium.pddl* and *problem1.pddl* to get the logical structure of the domain and problem files. After successfully running the two pddl files using planning.domains online planner, I got 98 sequential steps. This confirmed that the target state can be accomplished by transforming the initial state using the specified domain actions.



From the above screenshot, we can see a wide range of unstack, stack, pick-up, and put-down actions that are included in the output. All of them follow a specific domain restriction. For an example, look at the precondition section in the screenshot. This solution showed how the planner finds the best way to achieve the target block configuration with whatever staring block it has while respecting the logical restrictions specified in the domain. To achieve this, a methodical Deconstruction - Reconstruction pattern is used here.

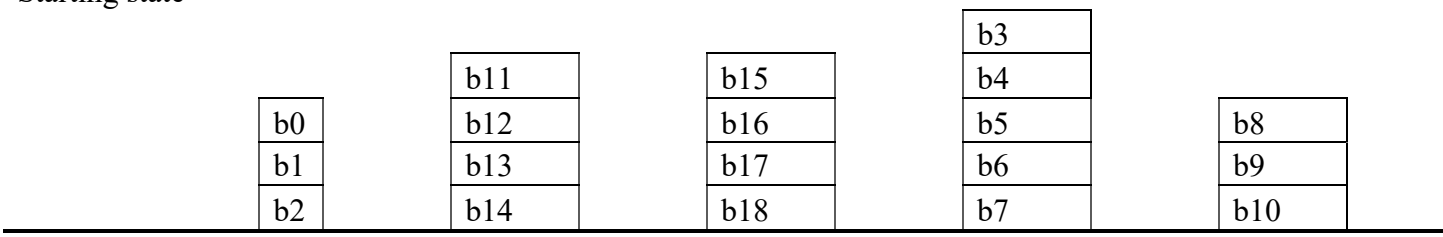
In the **Deconstruction phase,** the planner starts by determining which block must be unstacked in order to make room for rearranging. For example, in order to obtain the necessary blocks and disassemble the original column, the initial steps require a number of unstack operations.

In between the Deconstruction and Reconstruction phase there is a middle phase which is called **Temporary Placement.** We temporarily arrange the blocks on the table while making room or getting ready for the target/final stacking order. In this phase, put-down and pick-up actions are frequently used.

In the **Reconstruction phase,** the planner proceeds to the stacking phase after the blocks are readily available in the proper order. During this phase, it builds the final goal stack piece by piece, putting b on a, c on b, and so on until e is on top of the structure.

All these stages mirror a complicated planning process in which the agent must keep an internal eye on the state of the environment while predicting actions in the future. This is something very similar to the robotic planning we see in the real world where logics with certain conditions and proper foresight are essential. The below diagram is how the blocks are going be stacked on top of one another from the starting state and after the sequence of steps in the above screenshot are completed.

Starting state

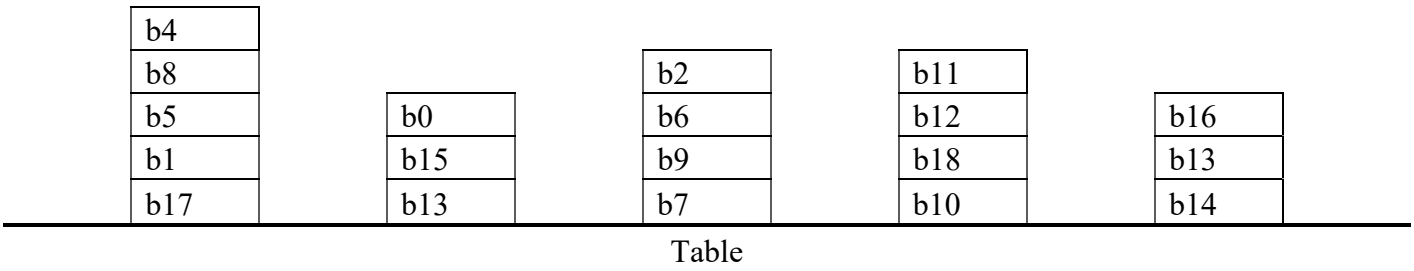


After executing the sequence of steps in the planner

A solid plan with the following performance stats were generated by the BFWS ff parser solver inn terms of computational metrics.

```
Total time: 0.041854
Nodes generated during search: 4791
Nodes expanded during search: 2619
Plan found with cost: 98
Fast-BFS search completed in 0.041854 secs
```

Goal state



The above diagram is the final structure of the blocks. This is the desired outcome in this problem file. When we look at the significance of performing, we can say that despite the 98 steps in the plan, the planner

- Never broke any rule such as hand occupancy or stacking on unclear blocks.

- It chose a step with minimal steps given the domain limits.
- Demonstrated the logical sequencing capabilities of PDDL solvers in symbolic reasoning

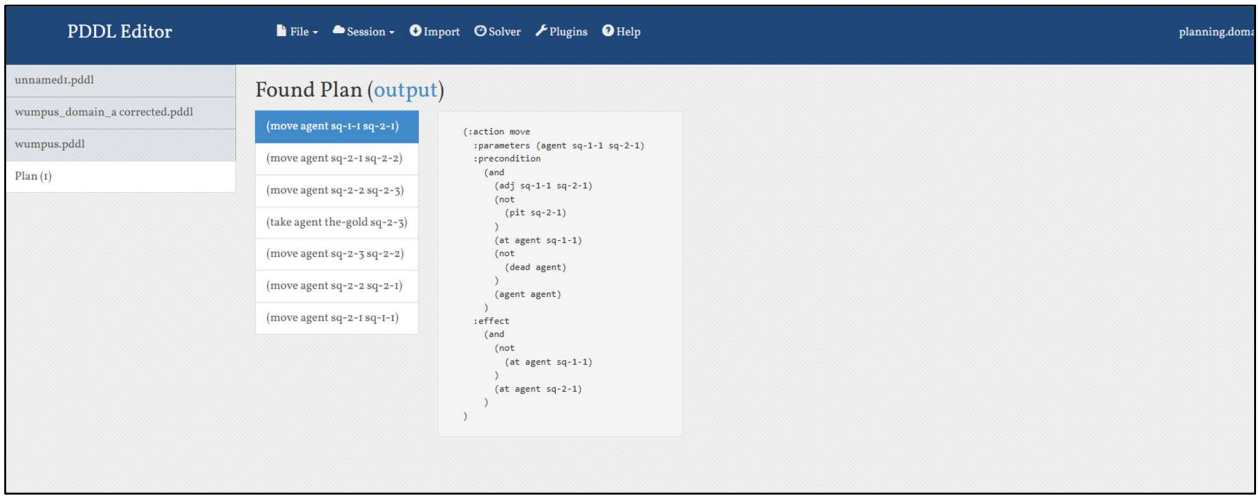
When we look at the significance of performing, we can say that despite the 98 steps in the plan, the planner

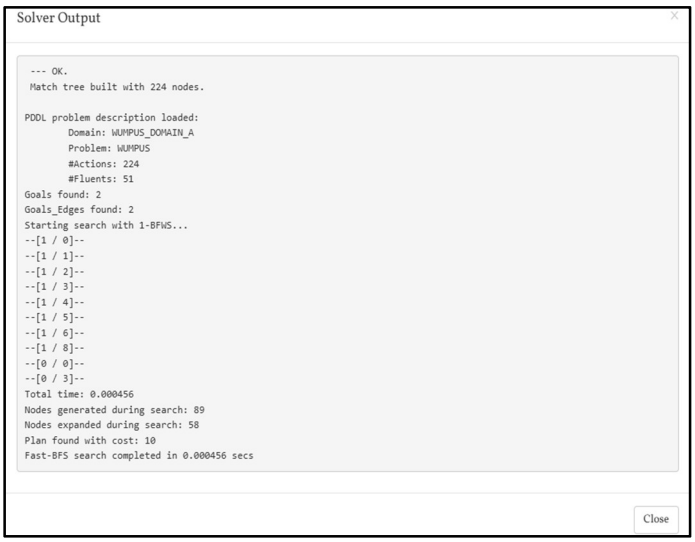
- Never broke any rule such as hand occupancy or stacking on unclear blocks.
- It chose a step with minimal steps given the domain limits.
- Demonstrated the logical sequencing capabilities of PDDL solvers in symbolic reasoning

Wumpus World

The planning.Domain online planner was used to effectively run the wumpus_domain_a.pddl and wumpus.pddl files. The planner produced a good sequence of steps with 7 logical operations. This confirmed that the agent could accomplish the given objective without going against any domain constraints and that the PDDL definitions made sense.

The screenshot below is a sample of the output given by the planner,





The planner effectively carried out the best course of action to address the issue.

1. **Move to Gold –** The agent will shoot the Wumpus if it got in the way as it safely made its way from its stating point to the gold's location, avoiding pits.

2. **Collect the Gold –** The agent does the 'take' action to get the gold as soon as they arrived at the location

3. **Return to the start –** After staying away from danger, the agent will go back to its starting position.

The accuracy and efficiency of the domain logic are confirmed by these planner statistics. The system clearly grasped the agent's movements, as evidenced by the modest node expansion and quick planning time. This confirms that restrictions like avoiding holes and only using the arrow when required are implemented correctly.

The goal state was shown in the code as follows.

```
48    (:goal (and (have agent the-gold) (at agent sq-1-1)))
```

The planner demonstrated the importance of automated planning in dangerous situations by efficiently creating a route that reduced risk and guaranteed survival. The pddl model smoothly and logically accomplished the project goals by clearly specifying the knowledge base, agent actions, and hazard logic. All things considered, this demonstrates the effectiveness of logic driven planning in AI contexts where achieving goals and surviving need careful balancing under predetermined norms and situations.

Wumpus World Constraint Solver Analysis –HD

Evaluation of the sensing constraint

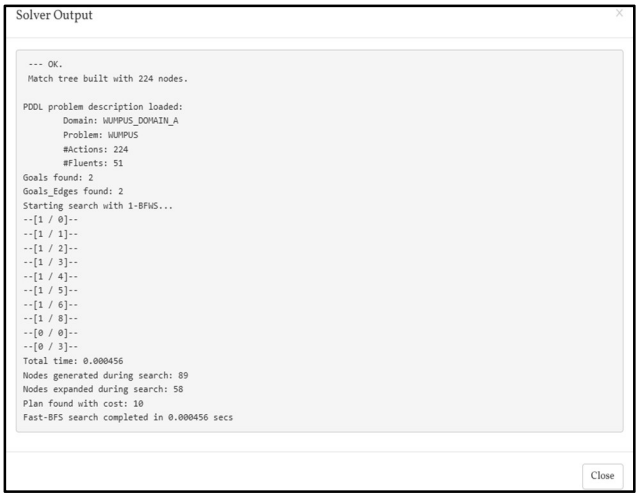
The effectiveness and security of the agent's navigation approach were directly impacted by the Sensing Constraint's integration.

- Path finding stability - it prevented blind moves where it could have fallen into pits or the Wumpus' lair by requiring the agent t sense each square before moving.

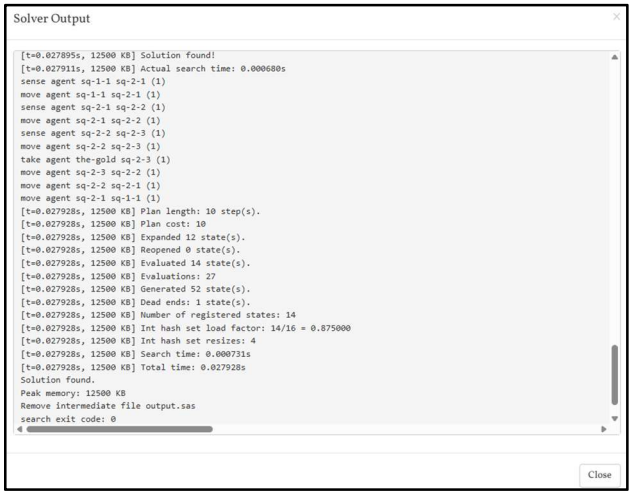
- Solver adaptations – This constraint improved optimization in the LAMA-First solver by reducing the number of backtracks because landmarks were detected before moving.

- Similarities to the real world – this behavior is similar to robotic navigation in the real world, where sensors are used to confirm safety before acting.

Feature	**BFWS FF Parser**	**LAMA-First**
Planning time	0.000456 sec	0.027928 sec
Nodes generated	89	52
Nodes expanded	58	12
Plan Length	10 steps	10 steps
Heuristic Used	Width-based + BFS	Landmark Sum + FF
Peak memory	Not given	12.2 MB
Dead ends met	Not visible	1



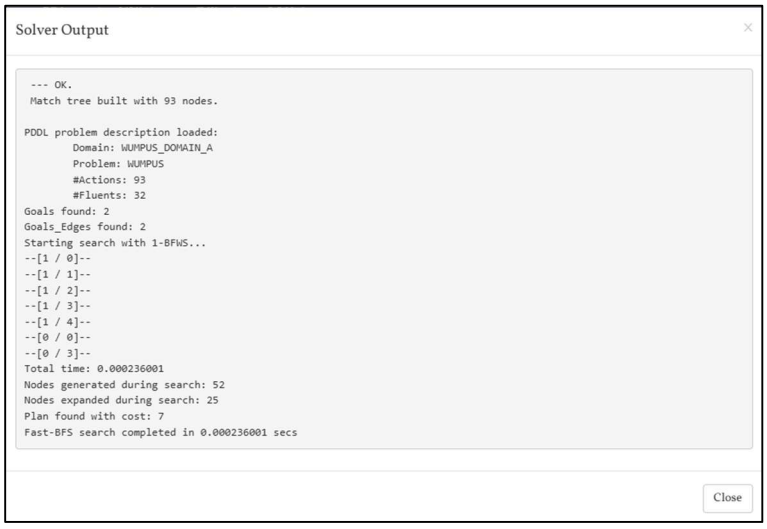
BFWS FF parser



LAMA-First

Constraint Impact Analysis

Metric	**Without constraint**	**With Sensing Constraints**
Plan steps	7	10
Nodes generated	52	89
Nodes expanded	25	58
Planning time (BFWS)	0.000236 sec	0.000456 sec
Solver stability	Stable	Stable
Plan effeciency	Shorter plan	More cautious, sensor dependant



Without the constrain output log

(The following files without the constraint will be submitted in a folder names ‘other’ for reference)

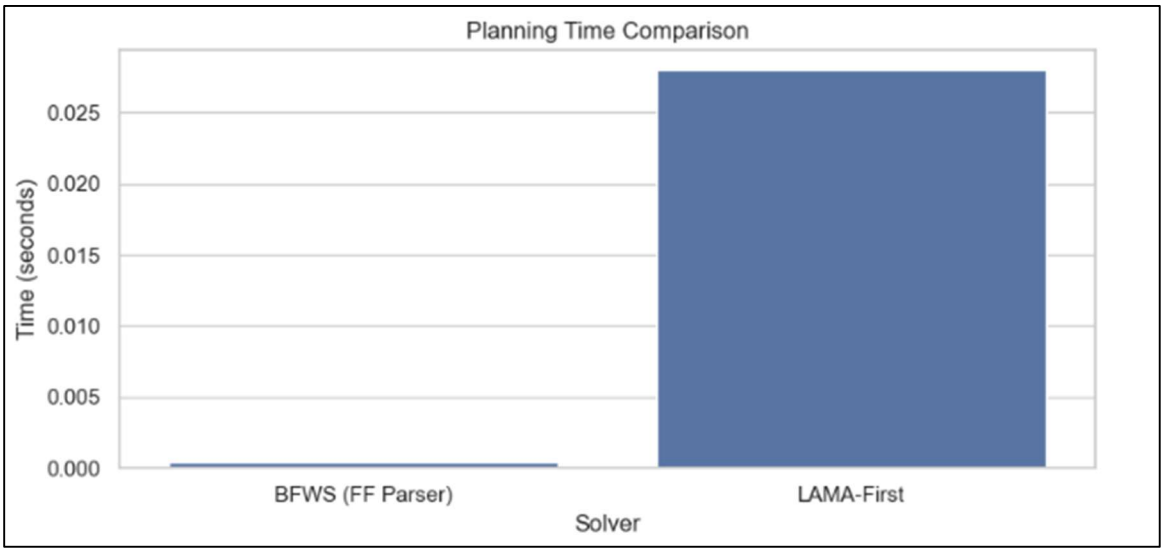
You can refer the with constrain output log in the previous section in **Results and Discussion**

Visual comparison

Using information gathered from the BFWS (FF Parser) and LAMA-First solvers output, I created three visual bar charts to better understand how each solver performed under the same problem settings. These comparisons can give us a good idea on search technique, efficiency, and planning speed.

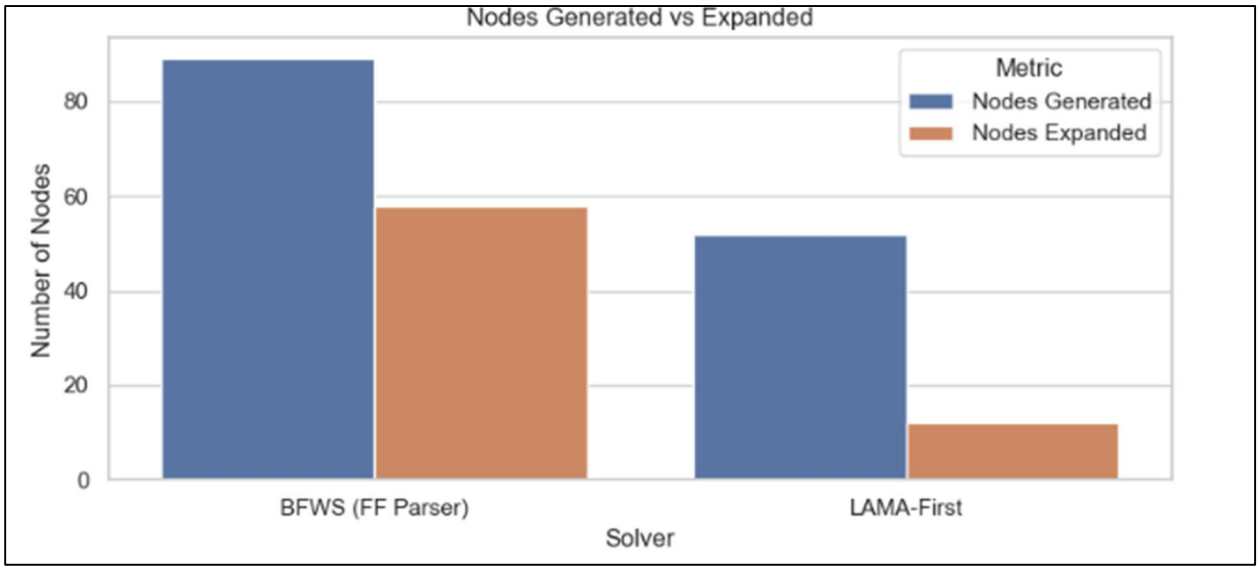
1. **Planning Time Comparison**

   This chart compares the execution time taken by each solver.



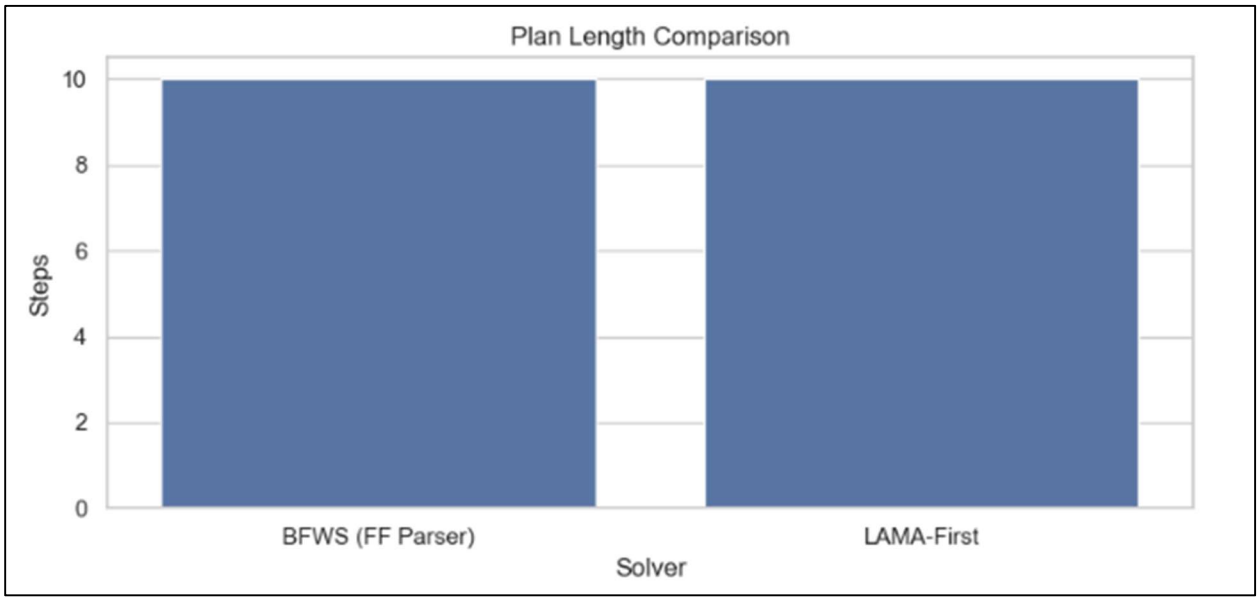
BFWS is super faster, but fast doesn't mean it could be all smart planning.

2. **Nodes Generates vs Nodes Expanded**



Better heuristic cutting with landmark and FF heuristics allowed LAMA-First to create and grow fewer nodes. This demonstrates increased planning effectiveness.

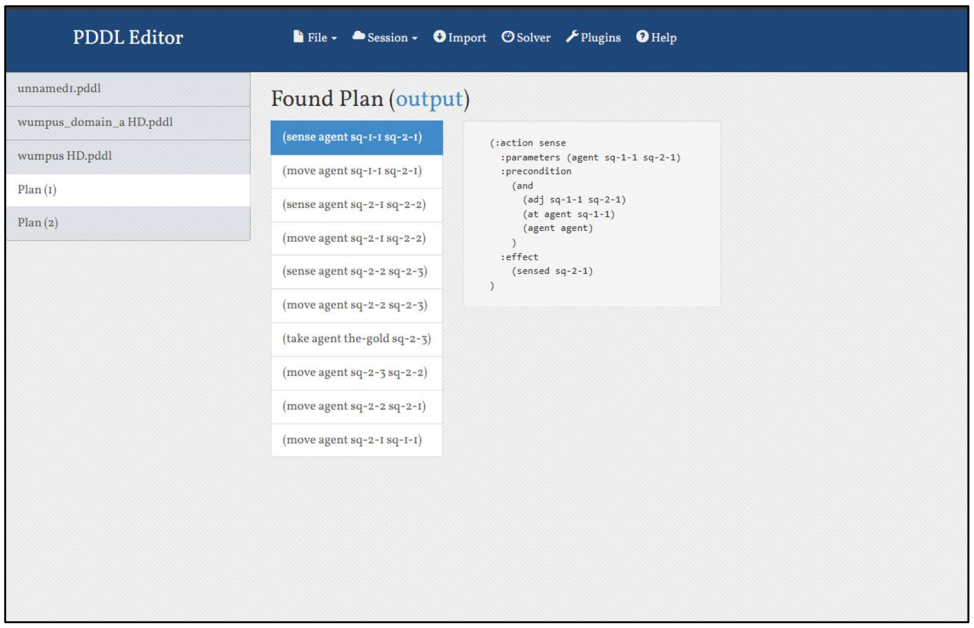
3. Plan length comparison



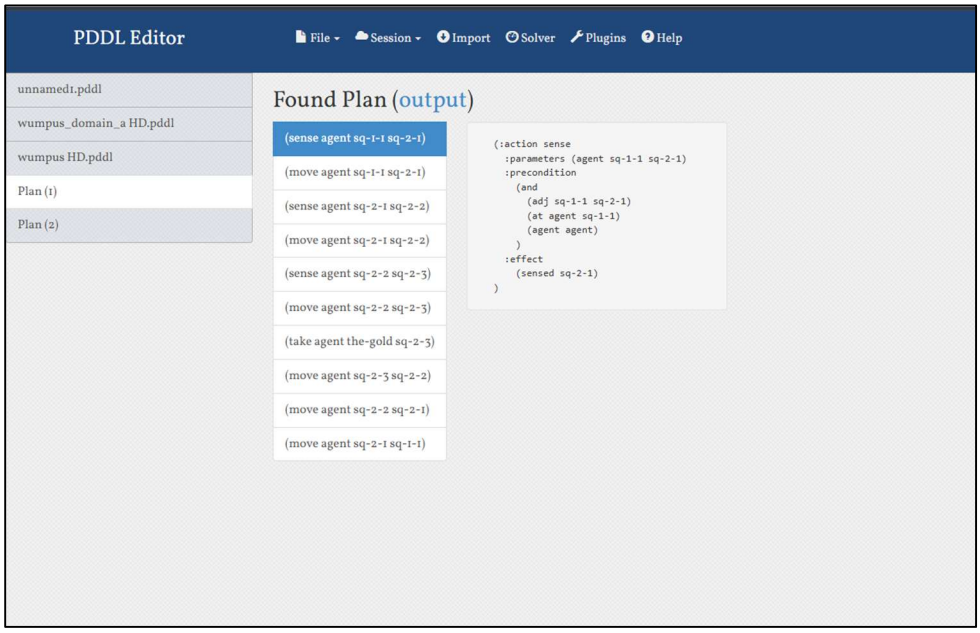
	Solver	Planning Time (s)	Nodes Generated	Nodes Expanded	Plan Length	Peak Memory (KB)
0	BFWS (FF Parser)	0.000456	89	58	10	NaN
1	LAMA-First	0.027971	52	12	10	12500.0

(The ipynb file used to generate these images will be submitted as well.)

The plans generated through *planning.domains* are shown below.



Using LAMA-First solver



Using BFWS ff parser version

Conclusion

Blocks World and Wumpus World were both executed successfully and this proved how useful PDDL (Planning Domain Definition Language) is in automated planning. Both situations successfully achieved their goal conditions by following agent based decisions making in controlled situations through safe path finding, constraint handling, and logical reasoning. Real world applications are reflected in these simulations, including autonomous path finding in grid based situations like automated delivery systems or disaster response, and robotic navigation in warehouses where blocks are constantly sorted and stacked.

In addition, the implementation of **sense** constraint in Wumpus world showed us the agent's ability to update its navigational approach based on environmental feedback, replicating real world auto-driving cars that depend on sensors to avoid obstacles ahead. Symbolic thinking and logic based decision making is the foundation for the development of intelligent agents in AI driven automation which aligns with Computer intelligence (CI) concepts.

The comparison of solvers between BFWS ff parser and LAMA-First showed how intelligent planning algorithms can greatly improve agent's performance in real world situations, emphasizing the importance of heuristic selection in maximizing path finding efficiency and execution time. There tests demonstrate how automated planning has the ability to completely transform sectors that need to make quick, smart decisions in both structured and unstructured environments.

In conclusion, the project showed the transformative potential of computational intelligence in resolving complex automation problems by offering practical experience with PDDL based planning, confirming key concepts in symbolic AI, and illustrating its practical implications in domains such as robotics, logistics, and autonomous navigation.

Acknowledgment

I want to express my sincere gratitude to the unit lecturers for their crucial advice and assistance during the educational process. A solid understanding of PDDL and its use in automated planning was developed thanks in large part to the Wednesday practical sessions and Tuesday online courses. I would especially like to thank the teaching staff for their patience and clarity in breaking down difficult ideas, which made learning much easier for me. In addition, the planning.domains platform and online resources provide a great basis for practical experience, which allowed me to successfully test and improve the PDDL solutions that were show cased in this project.

References

1. Tomsilver (no date) *pddlgym/pddlgym/pddl at master · tomsilver/pddlgym*. https://github.com/tomsilver/pddlgym/tree/master/pddlgym/pddl.

2. *PDDL Editor* (no date). https://editor.planning.domains/.